DetourTO — Stage 2 Implementation Report

EECS 3311 Software Design · York University · Fall 2026

Pratham Patel · 219467315 · github.com/prathamp18/EECS3311-DetourTO

November 2026

1. Project overview and implementation status

DetourTO is a disruption-aware trip planner for the Toronto Transit Commission (TTC). A rider types a trip in plain English ("York U to Union by 10, no streetcars, step-free please") or fills in a form. DetourTO returns several real itineraries, explains how they differ, and keeps them up to date when service alerts close stations or lines.

Users. The Stage 1 users are unchanged:

- daily TTC commuters (for example York students);
- riders who need step-free access;
- occasional riders who don't know the network;
- a developer or tester who replays recorded disruptions from the command line.

AI / LLM integration. The application calls **Google Gemini** (`gemini-flash-latest`, AI Studio free tier) through `GeminiAdapter`, which implements the `LLMClient` interface. Four agents extend the `TransitAgent` template:

Agent	What it does
`TripRequestAgent`	Turns plain English into a trip query, using the `find_stop` and `plan_journey` tools
`AlertInterpreterAgent`	Turns free-text TTC alerts into structured disruptions, using `find_stop` and `get_route_stops`
`ExplanationAgent`	Compares the computed options in plain language
`PreferenceExtractorAgent`	Picks lasting preferences out of a message ("never put me on the 29")

The agents take part in reasoning, tool use, memory and multi-step work. They do more than forward a prompt: every model answer is parsed by `ResponseParser` and checked by `GroundingValidator` before it is used. All stops, routes, times and itineraries come from the deterministic Java planner (RAPTOR), never from model text. Every model call, tool call and validation result is published on the `EventBus` and written to `trace.jsonl`.

Current working functionality. All 13 Stage 1 features (F01–F13) are implemented and run from both interfaces:

- **GUI:** JavaFX with 5 tabs (Trip Planner with map, Alerts, Arrivals, Commute & Profile, Benchmark), started with `java -jar target/detourto.jar gui`.
- **CLI:** the picocli command `detour`, with sub-commands `plan`, `alerts`, `arrivals`, `monitor`, `places`, `prefs`, `commute`, `replay`, `refresh-data`, `validate-feed`, `llm-check` and `gui`, and `--json` output.

The code base is about 12,100 lines of Java in 148 classes, plus 36 test classes with 179 JUnit 5 tests. Nine design patterns from the course are implemented (Section 3).

Limitations (stated honestly).

1. **Default data.** The default data is a small synthetic network (`data/sample/mini-gtfs`: 72 stops, 5 routes) and a recorded GTFS-Realtime feed, so the app and its videos are repeatable. The real TTC schedule can be downloaded with `refresh-data` and the live feed switched on in `.env`. Station grouping (step 18) was added for the real feed, but the full real schedule was only checked on the developer's machine, not in an automated test.
2. **Map.** The map is drawn on a JavaFX Canvas with no street basemap.

3. **Daylight-saving time.** Times are converted as the service date's midnight plus seconds. The GTFS "noon minus 12 h" rule for daylight-saving days is not handled.
4. **LLM dependence.** The AI features need a Gemini key. Without one, deterministic fallbacks are used: the form planner, rule-based follow-ups and preferences, and structured-alert parsing. Free-text alert interpretation and the natural-language explanation are then not available.
5. **Monitoring in the CLI.** Live monitoring in the CLI is a minute-by-minute replay, because a CLI process cannot wait for a real trip. The GUI monitors on the real clock.
6. **Stage 3 work.** KUMA agent tests and the full deterministic test suite are Stage 3 work.

2. Feature to implementation mapping

Every Stage 1 feature is implemented. No features were removed, and no new top-level features were added. Two feature-level additions inside existing features are listed in Section 4: station grouping for F13/F02 and map controls for F02. In the table, "Det." means deterministic, "AI" means LLM-based, and "Hybrid" means the LLM proposes and Java validates.

F01 — Natural-language trip request

Field	Entry
Stage 1 reference	F01 · Status: **Implemented** (Gemini instead of Claude/Bedrock; see Section 4)
Classes / files	`TripPlannerPanel` (Ask box) and `PlanCommand` (`detour plan "<text>"`) take the input. `TripController` is the facade. `TripRequestAgent` is the AI. `FindStopTool` and `PlanJourneyTool` (registered in `ToolRegistry`) are its tools. `ResponseParser` turns JSON into a `TripQuery`. `GroundingValidator` rejects any stop id that did not come from a tool result. `ClarificationRequest` is returned when a place is ambiguous.
Key methods	`TripController.planFromText()`, `TransitAgent.run()`, `TripRequestAgent.parseFinal()/validate()`, `GroundingValidator.validateQuery()`, `PlaceResolver.resolve()`
Patterns	Facade, Template Method, Adapter, Factory, Observer
Type	**Hybrid**

Flow: the user's text → `TripController.planFromText` → `TripRequestAgent.run` (system prompt, tool loop through `LLMClient.complete`) → `find_stop` resolves each place against `TransitNetwork` → the model returns a structured query → it is validated → `JourneyPlanner.plan` computes the options → `PlanResult` is shown in the GUI options list or printed by the CLI. Ambiguous places produce a clarification question (CLI exit code 4).

F02 — Multi-option journey planning

Field	Entry
Stage 1 reference	F02 · Status: **Implemented** (map changed; see Section 4)
Classes / files	`JourneyPlanner` runs a RAPTOR search over `TransitNetwork` / `NetworkView`. `GtfsStaticLoader` builds the network from GTFS. `Itinerary`, `RideLeg`, `WalkLeg` and `TripQuery` hold the trip data. `PlaceResolver` handles place names. `MapPanel` and `MapProjection` draw the chosen trip. `PlanCommand` and `TextResultWriter` / `JsonResultWriter` handle the CLI output.
Key methods	`TripController.planFromForm()`, `JourneyPlanner.plan(q, view, strategy)`, `MapPanel.drawItinerary()`
Patterns	Facade, Strategy
Type	**Deterministic**

Flow: From, To and a depart or arrive-by time → `planFromForm` resolves the places → the planner runs rounds of RAPTOR (one round per extra transfer) → up to 5 options, ranked by the chosen strategy → the list and the map in the GUI, text or `--json` in the CLI. The map zooms to the chosen trip and labels only the start, transfer and end stops.

F03 — Route preference strategies

Field	Entry
Stage 1 reference	F03 · Status: **Implemented** (interface extended)
Classes / files	The `RouteScoringStrategy` interface and `FastestStrategy`, `FewestTransfersStrategy`, `LeastWalkingStrategy`, `AccessibleStrategy`. `StrategyKind` names them. `ChangeStrategyCommand`

Field	Entry
	makes a strategy change undoable.
Key methods	`score(itinerary, query)`, `canBoard()` / `canAlight()` (used for pruning inside RAPTOR), `TripController.setStrategy()`
Patterns	Strategy, Command
Type	**Deterministic**

Flow: the "Rank by" box or `--strategy` → `setStrategy` runs a `ChangeStrategyCommand` → the planner uses that strategy's pruning and scoring. `AccessibleStrategy` refuses to board or alight at stops that are not step-free.

F04 — Service alert interpretation

Field	Entry
Stage 1 reference	F04 · Status: **Implemented**
Classes / files	`GtfsRealtimeAdapter` (own protobuf reader `GtfsRealtimeDecoder`) reads the TTC GTFS-RT alerts. `AlertService` turns each alert into a `Disruption`. Alerts with structured stop and effect fields are converted directly. Free-text alerts go to `AlertInterpreterAgent`, which uses `FindStopTool` and `RouteStopsTool`. `AlertsPanel` and `AlertsCommand` show the results.
Key methods	`AlertService.currentDisruptions()`, `AlertInterpreterAgent.interpret()`, `GroundingValidator.validateDisruption()`
Patterns	Adapter, Decorator, Template Method
Type	**Hybrid**

Flow: feed → `alerts()` → for each alert, either a structured mapping or the agent (with stop ids checked against the network) → a `Disruption` with kind, closed stops, confidence and a one-line summary → a table in the GUI, `detour alerts` in the CLI. Interpretations below 50% confidence are listed but not applied unless the user asks.

F05 — Disruption-aware re-routing

Field	Entry
Stage 1 reference	F05 · Status: **Implemented** (closure behaviour refined)
Classes / files	`TripController` builds a `NetworkView` with closed stops, closed line sections and outages (`TransitNetwork.withDisruptions` / `withClosures`). `JourneyPlanner` plans on that view. `AlertsPanel` has a "Re-route my trip around this" button. `MapPanel.highlightDisruptions()` marks closures.
Key methods	`planWithDisruptions()`, `rerouteAround(disruptionId)`, `NetworkView.isStopClosed()`
Patterns	Facade, Strategy
Type	**Hybrid** (the AI interprets the alert; routing is deterministic)

Flow: active disruptions are applied by default. `--avoid-disruption <id>` (or the GUI button) also applies a low-confidence one, and `--ignore-alerts` plans on the plain schedule for comparison.

F06 — Grounded explanation and comparison

Field	Entry
Stage 1 reference	F06 · Status: **Implemented**
Classes / files	`ExplanationAgent` writes the comparison, `Explanation` holds the text and the option ids it refers to, and `GroundingValidator.validateExplanation()` checks it. The "Explain" button and `--explain` start it.
Key methods	`TripController.explain()`, `ExplanationAgent.explain()`, `ExplanationAgent.fallback()`
Patterns	Template Method, Facade
Type	**Hybrid**

Flow: the computed options → the agent → the validator only accepts option ids, minute values and lines that exist in the options → if validation fails, a deterministic comparison is shown instead.

F07 — Live trip monitoring

Field	Entry
Stage 1 reference	F07 · Status: **Implemented**
Classes / files	`TripMonitorService` polls the feed. `ActiveTrip` is the context, with the states `PlannedState`, `OnTrackState`, `AtRiskState`, `ReroutedState` and `CompletedState`. `TripRisk` describes what is wrong.

Field	Entry
	The GUI uses the “Start trip” / “Accept reroute” buttons; the CLI uses `MonitorCommand`.
Key methods	`TripController.startTrip()`, `TripMonitorService.poll()`, `ActiveTrip.update()`, `TripState.handle()`, `acceptReroute()`
Patterns	State, Observer, Facade
Type	**Deterministic** (alert interpretation inside it may use AI)

Flow: start a trip → on every poll the current state checks the remaining legs against disruptions, delays (missed connections) and the arrive-by deadline → it moves to At risk and publishes an event → the GUI badge and the CLI log update → if the user accepts a reroute, the trip moves to Rerouted, and when the last leg ends it moves to Completed.

F08 — Stop arrivals board

Field	Entry
Stage 1 reference	F08 · Status: **Implemented** (arrivals moved to the application layer)
Classes / files	`ArrivalBoard` combines the schedule with the `RealtimeSnapshot` delays. `CachingRealtimeFeed` and `ResilientRealtimeFeed` (with `CircuitBreaker`) wrap `GtfsRealtimeAdapter`. `ArrivalsPanel` and `ArrivalsCommand` show the board.
Key methods	`TripController.getArrivals()`, `ArrivalBoard.arrivals()`
Patterns	Decorator, Adapter
Type	**Deterministic**

Flow: stop name → the next departures, with the live delay shown where the feed has one. If the feed fails, the circuit breaker falls back to scheduled times.

F09 — Saved places and preference memory

Field	Entry
Stage 1 reference	F09 · Status: **Implemented**
Classes / files	`RiderProfile` and `PreferenceUpdate` hold the profile. `ProfileRepository` has an SQLite version (`SqliteProfileRepository`, on `SqliteDatabase`) and an in-memory version. `PreferenceExtractorAgent` is the AI, with `PreferenceRules` as the offline fallback. `PlaceResolver.setSavedPlaces()` lets “home” work anywhere. The GUI uses `CommutePanel`; the CLI uses `PlacesCommand` and `PrefsCommand`.
Key methods	`savePlace()`, `learnPreferences()`, `updateProfile()`, `applyProfile()`
Patterns	Template Method, Facade
Type	**Hybrid**

Flow: “never put me on the 29” → the agent (or the rules) produces a `PreferenceUpdate` → it is validated against real routes → it is saved in SQLite → it is applied to every new query (avoided routes, the step-free strategy, the default strategy).

F10 — What-if follow-ups with undo/redo

Field	Entry
Stage 1 reference	F10 · Status: **Implemented** (one command added)
Classes / files	`QueryCommand` has 5 implementations: `ChangeDepartureCommand`, `AvoidRouteCommand`, `AvoidModeCommand`, `ChangeStrategyCommand` and `ChangeDestinationCommand`. `CommandHistory` holds the undo and redo stacks. `QueryCommandFactory` builds commands. `FollowUpIntent` / `IntentKind` describe the request. `TripRequestAgent.interpretFollowUp()` is the AI, and `FollowUpRules` is the offline fallback. The GUI uses the Ask box and the Undo/Redo buttons; the CLI uses `--then`.
Key methods	`followUp()`, `undo()`, `redo()`, `CommandHistory.run()`
Patterns	Command, Factory, Template Method
Type	**Hybrid**

Flow: “leave 15 min later” → an intent (from the AI, validated, or from the rules) → the factory builds a command → `run()` changes the query and re-plans → `undo` restores the previous query.

F11 — Commute watch and leave-by alerts

Field	Entry
Stage 1 reference	F11 · Status: **Implemented**
Classes / files	`Commute`, `CommuteRepository` (SQLite and in-memory versions) and `CommuteScheduler` do the work. `CommutePanel` checks every minute; the CLI uses `CommuteCommand`.
Key methods	`saveCommute()`, `CommuteScheduler.today()`, `checkDue()`, `extraMinutes()`
Patterns	Facade, Observer
Type	**Deterministic** (uses the disruptions from F04)

Flow: the saved commute → today's arrive-by plan with current disruptions → the leave-by time and how much earlier than usual to leave → a warning `--warn` minutes before, once per day.

F12 — Disruption replay and benchmark

Field	Entry
Stage 1 reference	F12 · Status: **Implemented**
Classes / files	`ReplayScenario` and `ScenarioLoader` read `scenarios/*.json`. `ReplayRealtimeFeed` plays back the recorded alerts. `BenchmarkRunner` produces a `BenchmarkReport`. The GUI uses `BenchmarkPanel`; the CLI uses `ReplayCommand`.
Key methods	`runReplay()`, `BenchmarkRunner.runAll()`
Patterns	Adapter (replay feed), Observer (metrics)
Type	**Hybrid**

Flow: each scenario → interpret its alerts → plan around them → compare the interpreted closures with the closures that really happened → report the extra travel time, grounding violations, LLM calls and latency.

F13 — Transit data refresh and validation

Field	Entry
Stage 1 reference	F13 · Status: **Implemented** (station grouping added)
Classes / files	`OpenDataDownloader` handles the Toronto Open Data CKAN package and the ZIP download. `GtfsStaticLoader.validate()` and `FeedValidationReport` check the feed, and `groupStations()` groups platforms into stations. The GUI uses the Benchmark tab's "Refresh transit data" button; the CLI uses `RefreshDataCommand` and `ValidateFeedCommand`.
Key methods	`TripController.refreshTransitData()`, `OpenDataDownloader.download()`, `GtfsStaticLoader.load()`
Patterns	Facade
Type	**Deterministic**

Flow: download → validate → the new network is swapped in only if it is valid; otherwise the old one is kept. TTC platforms named "… Station" are grouped, so "Don Mills" is found.

3. Implemented design patterns

Nine distinct course patterns are in the code. Stage 1 listed eight, plus Factory as a supporting pattern.

Pattern	Problem it solves	Participants (roles)	Source	Matches Stage 1?
Strategy	Different riders rank trips differently, and accessibility must affect the search itself, not only the ranking	`RouteScoringStrategy` (Strategy), `FastestStrategy`, `FewestTransfersStrategy`, `LeastWalkingStrategy`, `AccessibleStrategy` (Concrete Strategies), `JourneyPlanner` (Context)	`domain/*Strategy.java`; `JourneyPlanner.plan()` calls `canBoard`, `canAlight` and `score`	Yes; `canBoard/canAlight` added so RAPTOR can prune
Facade	The GUI and the CLI must not know about the planner, agents, feeds or storage	`TripController` (Facade); subsystems: `JourneyPlanner`, the agents, `AlertService`, the repositories, `TripMonitorService`	`application/TripController.java`	Yes
Adapter	Gemini REST and GTFS-RT	`LLMClient` (Target) ← `GeminiAdapter`	`agent/`	Changed:

Pattern	Problem it solves	Participants (roles)	Source	Matches Stage 1?
	protobuf don't match the interfaces the app uses	(Adapter) → Gemini `generateContent` (Adaptee); `RealtimeFeed` (Target) ← `GtfsRealtimeAdapter` → TTC protobuf feed	`GeminiAdapter.java`, `infrastructure/GtfsRealtimeAdapter.java`	`BedrockClaudeAdapter` → `GeminiAdapter`; same role
Template Method	Every agent runs the same loop (prompt → tool calls → parse → validate → retry or fall back), with different prompts, tools and parsing	`TransitAgent.run()` (template); hooks `systemPrompt()`, `toolNames()`, `parseFinal()`, `validate()`, `onInvalid()`; subclasses `TripRequestAgent`, `AlertInterpreterAgent`, `ExplanationAgent`, `PreferenceExtractorAgent`	`agent/TransitAgent.java`	Yes
Decorator	Caching and failure handling must wrap any feed without changing it	`RealtimeFeed` (Component), `GtfsRealtimeAdapter` (Concrete Component), `RealtimeFeedDecorator` (Decorator), `CachingRealtimeFeed` and `ResilientRealtimeFeed` + `CircuitBreaker` (Concrete Decorators)	`infrastructure/*RealtimeFeed.java`	Yes
Observer	The trace log, CLI `--verbose`, GUI panels and monitor alerts all react to the same events without coupling	`EventBus` (Subject), `EventListener` (Observer), `TraceLogger`, `ConsoleNotifier`, GUI panels (Concrete Observers), `TransitEvent`	`application/EventBus.java`, `TraceLogger.java`	Yes; agents report through `AgentStepListener`
Command	Follow-up changes must be undoable and redoable	`QueryCommand` (Command), 5 concrete commands, `CommandHistory` (Invoker), `TripQuery` (Receiver state), `TripController` (Client)	`application/*Command.java`, `CommandHistory.java`	Yes; `AvoidModeCommand` added
State	A monitored trip behaves differently when planned, on track, at risk, rerouted or completed	`ActiveTrip` (Context), `TripState` (State), `PlannedState`, `OnTrackState`, `AtRiskState`, `ReroutedState`, `CompletedState`	`application/ActiveTrip.java`, `*State.java`	Yes
Factory	Choosing the LLM provider and building the right command from an intent	`LLMClientFactory.create(cfg)` (gemini or scripted), `QueryCommandFactory.create(intent, query)`	`agent/LLMClientFactory.java`, `application/QueryCommandFactory.java`	Yes (it was a supporting pattern in Stage 1)

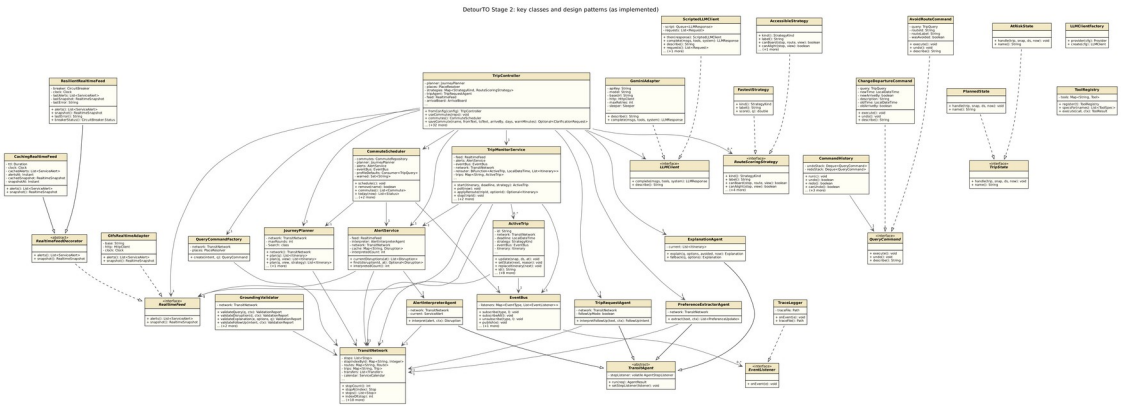
Why the patterns help.

- **Strategy:** adding a ranking means adding one class.
- **Facade:** the GUI and the CLI share one API, so each feature exists only once.
- **Adapter:** switching from Claude/Bedrock to Gemini changed only one class.
- **Template Method:** all four agents share the same validation and fallback loop.
- **Decorator:** caching and the circuit breaker are tested on their own.
- **Observer:** the trace file used for KUMA in Stage 3 is just another listener.
- **Command:** undo and redo are a pair of stacks.
- **State:** each trip state's rules live in their own class.

Updated UML class diagrams. These diagrams were generated from the actual Stage 2 source with Graphviz. They are in `docs/diagrams/stage2/` (PNG and SVG):

- an overview of the pattern participants (below);
- one diagram for each layer: presentation (CLI), presentation (GUI), application, agent, domain and infrastructure.

Hollow triangles show inheritance (solid) and interface implementation (dashed). Plain arrows are associations from fields, with their multiplicity.



Key classes and design patterns as implemented

Use-case and sequence diagrams. The 19 use cases and 6 actors are unchanged, so the Stage 1 use-case diagram is still accurate. The sequence diagrams are also still accurate in their messages. The only change is that the `BedrockClaudeAdapter` lifeline is now `GeminiAdapter`, behind the same `LLMClient.complete()` call. In SD05, `detour monitor` is replayed minute by minute in the CLI. Both changes are recorded in Section 4.

4. Changes and rationale

Each row reads: Original design → Implemented change → Reason → Impact.

Original design (Stage 1)	Implemented change	Reason	Impact
LLM = Claude on Amazon Bedrock (`BedrockClaudeAdapter`)	Google Gemini via `GeminiAdapter`; `LLMClientFactory` picks `gemini` or `scripted` from `LLM_PROVIDER`	Every Bedrock per-model daily token quota on the new AWS account was 0 and could not be raised; Gemini has a free tier	Only the adapter class differs; the agents only see `LLMClient` (Adapter pattern)
`Message(role, content)`	Also carries tool calls, tool-call id and name, and opaque provider data; `Role.TOOL` added	A tool-use loop needs tool turns; Gemini requires its "thought signatures" to be sent back	Same class, more fields
`LLMClient.complete` only	Added `describe()` and `LLMException` kinds (AUTH, RATE_LIMIT, BAD_REQUEST, UNAVAILABLE, CONFIG)	Clear errors, and the provider/model is recorded in the trace	The CLI says why a call failed; retries on 503
`StopTime` holds `LocalTime`	`Trip` stores int arrays (seconds after midnight); `StopTime` is a view	GTFS times pass 24:00; the real feed has millions of stop times	Less memory; faster planner
`TransitNetwork.withDisruptions(ds)`	Added `withClosures(stops, routes, outages)`; a closure naming a route and stops also closes that line section	Found in testing: trains were still routed through a closed stretch	Correct reroutes for "no service between A and B"
— (new class)	`RoutePattern`	RAPTOR scans trips that share a stop sequence	Not in the Stage 1 class diagram
`RouteScoringStrategy: admissible, score(i)`	`canBoard` / `canAlight` (prune inside RAPTOR), `score(i, q)`, `kind()`	Accessible trips must be found, not just ranked; scoring needs the query time	Interface has 2 extra methods
`TripQuery` fields	Added `avoidModes`	F01's "no streetcars" had no field in the Stage 1 class diagram	`--avoid-mode` option
`JourneyPlanner.plan(q, ds, s)`	`plan(q, view, s)` plus shortcuts	The controller builds the network view once	Same behaviour
`DetourCli` has `main()`	`Main` is a thin entry point; `DetourCli` is the picocli root with one class per sub-command	Single responsibility	Same CLI
`PlaceResolver` depends on `ProfileRepository`	`PlaceResolver` gets a saved-place lookup from	Keeps the domain free of infrastructure	"home" works everywhere

Original design (Stage 1)	Implemented change	Reason	Impact
	`TripController`		
`GtfsRealtimeAdapter` uses `gtfs-realtime-bindings`	Own small protobuf reader (`ProtoReader`, `GtfsRealtimeDecoder`)	Avoids a large dependency; only a subset of GTFS-RT is needed	Same adapter interface; decoder unit-tested
`RealtimeFeed.arrivals(stopId)`	Arrivals computed in `ArrivalBoard` (schedule + live delays); the feed has `alerts()` and `snapshot()`	Arrivals need the static schedule; decorators should only wrap network calls	Same F08 behaviour
`TransitAgent` holds the `EventBus`	`TransitAgent` reports through `AgentStepListener`; `TripController` forwards to the `EventBus`	Avoids a package cycle between agent and application	Observer still drives the trace, CLI and GUI
— (new)	`RouteStopsTool` (`get_route_stops`)	Free-text alerts name stops in words; the model needs the route's stop ids to ground them	Extra agent tool
— (new)	Recorded sample GTFS-RT feed and replay scenarios	Live TTC trip ids only match the real schedule; tests and videos need repeatable data	Default `TTC_FEED_BASE_URL=data/sample/realtime`
`Disruption`, `Arrival`, `ReplayScenario` fields	Added `summary`; `tripId`, `headsign`, scheduled/realtime flags; scenario `from`, `to`, `strategy`, `arriveBy`	Needed by the UI and for scenarios that run without a key	—
4 follow-up commands	`AvoidModeCommand` added; `ChangeDepartureCommand` also switches between depart-at and arrive-by	"no streetcars" and "be there by 10" are common follow-ups	5 commands
`IntentKind` not detailed	8 kinds plus the offline `FollowUpRules` fallback	F10 must work even if the LLM answer fails validation	Follow-ups never block
`setStrategy` changes the query directly	Runs a `ChangeStrategyCommand`	Stage 1 says strategy changes are undoable	Undo covers strategy
`PreferenceUpdate` in the agent package	Moved to the domain	`RiderProfile.apply()` would otherwise make the domain depend on the agent package	No package cycle
— (new)	`PreferenceRules`; preference learning only runs when a message states a lasting preference	Avoids an extra LLM call on every request; works without a key	Fewer LLM calls
`ScriptedLLMClient` throws when out of responses	Throws `LLMException(UNAVAILABLE)`	Found in testing: offline mode crashed `plan`	Agents fall back gracefully
Monitor polls every 30 s	Same in the GUI (plus a fast-replay option); the CLI replays minute by minute	A CLI process cannot wait for a real trip	`--accept-reroute`, `--plan-with-alerts`
`ActiveTrip` states	Missed-connection risk is kept until arrival; reroute options exclude the trip already being taken	Found in testing: the trip went back to on-track after the transfer time, and the "reroute" was the same bus	Correct at-risk behaviour
`detour monitor <itineraryId>`	`detour monitor --from -- to [--depart | --arrive-by] --option opt-N`	Each CLI run is a new process with no saved ids	Same feature
`Commute` origin and destination	Stored as stop ids in SQLite	Survives restarts and data refreshes	Commutes whose stops vanish are skipped
`OpenDataDownloader(url)`	Accepts a direct .zip URL or the CKAN package URL; `--file` loads a local zip	The portal's file URL changes; offline demo	`TTC_GTFS_URL` in `.env.example`
Station lookup by name	Platforms named "… Station" without a parent are grouped into generated stations (`groupStations`)	The real TTC GTFS has no parent stations, so "Don Mills" was not found	Place names resolve on the real feed
`MapPanel` with WebView + Leaflet/OpenStreetMap	Map drawn on a JavaFX Canvas (`MapProjection`)	Works offline, with no web tiles or JavaScript; closures can be drawn directly	No street basemap

Original design (Stage 1)	Implemented change	Reason	Impact
`MapPanel` labels every station	Zooms to the chosen trip; labels only start, transfers and end (plus subway names); other names on hover; zoom, pan, Fit trip and Whole map controls	On the full TTC network the map was unreadable	Readable at city scale
`MainWindow` with 6 panels	5 tabs; Commute and Profile share a tab; the map is inside Trip Planner	Simpler navigation	Same features
Build	SQLite JDBC, SLF4J no-op, JavaFX added to `pom.xml`; module-info excluded from the fat jar; `Enable-Native-Access`	Needed by SQLite and the GUI; removes JDK warnings	One `java -jar target/detourto.jar`
Factory (supporting pattern)	Counted as an implemented pattern	Both factories solve real choices (provider, command type)	9 patterns instead of 8

5. AI tools and collaboration log

Tool / platform	Model	Used for	Tokens	Money cost
Claude app (Cowork, coding assistant)	Claude Opus 5.5 (`claude-opus-5-5`)	Code generation for all 20 implementation steps: Maven setup, GTFS loader, RAPTOR planner, strategies, facade and CLI, LLM layer, the 4 agents and tools, GTFS-RT feed and decorators, alerts, explanations, replay, commands with undo, SQLite profile, trip states, commutes, data refresh, JavaFX GUI, station grouping and the map. Also used for JUnit tests, debugging, the README and drafting this report.	Measured for the last working session (map, README, report): about 6.8 M cached input tokens read, 0.19 M cache-write tokens and 29.5 K output tokens (about 1.27 M input-token equivalents). Earlier sessions were not metered per token.	No per-token charge (covered by a Claude subscription)
Google AI Studio / Gemini API (runtime LLM of the app)	`gemini-flash-latest` (fallback `gemini-flash-lite-latest`)	The 4 agents at runtime and during manual testing: trip requests, alert interpretation, explanations, preference extraction	**[fill in: total from `trace.jsonl`, see AI-TOOLS-LOG.md]**	$0 (free tier)
Amazon Bedrock (tried, then dropped)	Claude Sonnet on Bedrock	First LLM integration attempt	0 (quota was 0)	$0

How I worked with the AI. I gave Claude one feature at a time, in the order of the Stage 1 design. For every step I did the same things:

1. unzipped the generated changes;
2. ran `mvn -q package` (which compiles and runs the tests);
3. ran the new CLI command and the GUI;
4. read the diff;
5. committed and pushed it myself.

When something was wrong, I described what I saw, and the fix went into the next commit. Representative interactions:

What I asked / reported	What came back	My check
"Implement the RAPTOR planner from the Stage 1 design with arrive-by and closures"	`JourneyPlanner`, `Itinerary`, legs, 15 tests	Ran `detour plan` on the sample network; compared times with the GTFS
Bedrock calls failed (daily quota 0); "just use Gemini, don't change diagrams"	`GeminiAdapter` with function calling and thought signatures, retry on 503	`detour llm-check --tools` on my key
"`plan` crashes without a key"	`ScriptedLLMClient` throws `LLMException`; agents fall back	Ran offline with `LLM_PROVIDER=scripted`
The monitor said "risk cleared" after a missed connection, and the reroute offered the same bus	Missed-connection risk kept until arrival; reroute excludes the current trips	`detour monitor ... --accept-reroute`
SQLite warnings ("restricted method") on my JDK	`Enable-Native-Access` in the jar manifest and surefire	`mvn package` with no warnings

What I asked / reported	What came back	My check
“The GUI can’t find Don Mills on the real TTC data”	Station grouping in `GtfsStaticLoader` + test	Real feed: Don Mills → Pioneer Village
“Too much text on the map when a trip is chosen”	Fit to trip, key-stop labels only, hover names, zoom/pan	Planned trips in the GUI on the sample and real data

The full log, with the command used to count Gemini tokens, is in `AI-TOOLS-LOG.md`. No API key appears in the code, the report, the screenshots or the videos. `.env` is git-ignored, and `.env.example` holds placeholders only.

6. Development and verification summary

Weekly commit summary

Repository: https://github.com/prathamp18/EECS3311-DetourTO (dates are Toronto time).

Period 1 (Oct 20–26): no code commits. I started implementation on Oct 28, so this period does not meet the minimum, and I accept the 5-point deduction. No commits were backdated.

Period 2 (Oct 27–Nov 2): 11 commits.

Date	Commit	Message
Oct 28	f51d2dd	Set up Maven project, AppConfig with .env loading, and run instructions
Oct 28	f531065	Load GTFS static feeds into the transit network model
Oct 28	0f2dab5	Add RAPTOR journey planner with itineraries, closures and arrive-by
Oct 28	c9d7fe2	Add route scoring strategies and use them inside the planner
Oct 28	c1fb911	Add TripController facade, place resolver and the detour plan command
Oct 29	6dd2383	Add LLM client layer with Gemini adapter, scripted client and factory
Oct 29	9f954f1	Add trip-request agent for plain-English planning (F01)
Oct 29	da4be7e	Add GTFS-Realtime feed with caching and circuit breaker, and the arrivals board (F08)
Oct 31	4ea5a89	Interpret service alerts and plan around disruptions (F04, F05)
Oct 31	f456d88	Add grounded explanations (F06) and the event bus with trace logging
Oct 31	9596f6d	Add disruption replay and benchmark (F12)

Period 3 (Nov 3–9): 7 or more commits.

Date	Commit	Message
Nov 4	5c4eaec	Add what-if follow-ups with undo and redo (F10)
Nov 4	b613214	Add saved places and learned preferences in SQLite (F09)
Nov 4	804639e	Add live trip monitoring with trip states (F07)
Nov 4	c58dc86	Add commute leave-by alerts and transit data refresh (F11, F13)
Nov 7	7f2d699	Add JavaFX window with trip planner and map
Nov 7	6ee456d	Add alerts, arrivals, commute and benchmark tabs
Nov 7	d696095	Group TTC platforms into stations so place names find them
Nov 7–8	see history	Make the map readable on the full TTC network

Checks performed

- **Unit tests.** `mvn package` runs 179 JUnit 5 tests, with no network needed. They cover:
  - the GTFS loader and validation, and station grouping;
  - RAPTOR, including transfers, arrive-by, closures and closed line sections;
  - all 4 strategies;
  - place resolution and clarification;

  - the Gemini adapter against a local fake HTTP server (tool calls, thought signatures, 429/503 retries, auth errors);
  - every agent with scripted LLM answers, including invalid answers and fallbacks, and the grounding validator;
  - the protobuf decoder, the cache and the circuit breaker;
  - the alert service, the command history and every command;
  - the SQLite repositories, trip states and monitoring, the commute scheduler, the downloader, replay metrics and the map projection.
- **Manual checks.** Every CLI command was run on the sample data and with my Gemini key (`detour llm-check --tools`). The GUI was checked tab by tab, including plan → explain → start trip → accept reroute. The real TTC schedule was loaded with `refresh-data` and `validate-feed`.
- **Security checks.** `git status` never lists `.env`, `data/local/` or `*.db`. A search of the repository for key patterns finds nothing.

Important defects found and corrected

1. **Closed line sections.** Trains still ran through a closed section of a line. Fixed in `withClosures`.
2. **Offline crash.** The scripted LLM crashed `plan` when it ran out of answers. It now throws `LLMException` and the agents fall back.
3. **Monitor risk and reroute.** The monitor cleared a missed-connection risk too early, and the reroute offered the same bus. Both fixed.
4. **GUI errors.** The GUI threw a NullPointerException when there was no clarification. An alert that had not been interpreted was labelled “AI (validated)”. The strategy box showed enum names instead of labels. All fixed.
5. **XML comment in `pom.xml`.** A comment containing `--` broke Maven parsing. Fixed.
6. **SQLite warnings.** SQLite printed native-access warnings on JDK 21+. Fixed with the manifest and surefire settings.
7. **Real TTC stations.** On the real TTC feed, “Don Mills” was not found because there are no parent stations. Fixed with station grouping (step 18).
8. **Map clutter.** The map was unreadable with the full network. Fixed with fit-to-trip, key-stop labels and hover names (step 19).

Remaining limitations

These are the limitations listed in Section 1. KUMA agent tests and broader deterministic testing are planned for Stage 3. The `trace.jsonl` file and the `--json` output are already in place for them.

7. Demonstration index

Each feature has its own short video showing the GUI and the CLI, plus the error or alternative case where one applies. All videos are unlisted on YouTube and can be opened without an access request. No API key is shown in any video.

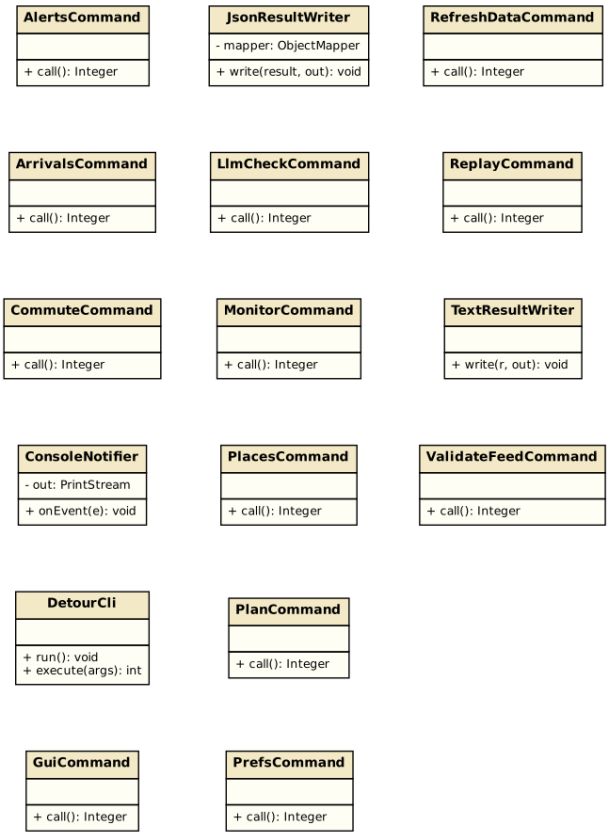
Feature	What the video shows	Video link
F01 Natural-language trip request	GUI Ask box “York U to Union by 10, no streetcars”; CLI `detour plan "..." --verbose`; ambiguous place → clarification	**[paste link]**
F02 Multi-option journey planning	GUI form Keele → Finch, options list, map fit and hover; CLI `plan --from --to --arrive-by --json`	**[paste link]**
F03 Route preference strategies	Rank by fastest / fewest transfers / accessible; CLI `--strategy accessible` avoids St Patrick	**[paste link]**
F04 Service alert interpretation	Alerts tab (structured vs AI-interpreted, confidence); CLI `detour alerts`	**[paste link]**
F05 Disruption-aware re-routing	“Re-route my trip around this”, closures on the map; CLI `--avoid-disruption` vs `--ignore-alerts`	**[paste link]**
F06 Grounded explanation	Explain button; CLI `--explain`; deterministic fallback without a key	**[paste link]**
F07 Live trip monitoring	Start trip → At risk → Accept reroute → Completed; CLI `detour monitor --accept-reroute`	**[paste link]**
F08 Stop arrivals board	Arrivals tab for Dufferin with live delays; CLI `detour arrivals "Dufferin" -n 8`	**[paste link]**

Feature	What the video shows	Video link
F09 Saved places and preferences	Save “home”; “never put me on the 29”; CLI `places add`, `prefs --learn`	**[paste link]**
F10 What-if follow-ups with undo/redo	“leave 15 min later”, “avoid line 1”, Undo/Redo; CLI `--then ... --then undo`	**[paste link]**
F11 Commute watch and leave-by alerts	Add a commute, leave-by time with a disruption; CLI `commute add` / `commute check`	**[paste link]**
F12 Disruption replay and benchmark	Benchmark tab; CLI `detour replay scenarios`	**[paste link]**
F13 Transit data refresh and validation	“Refresh transit data”; CLI `refresh-data --file`, `validate-feed`; invalid zip rejected	**[paste link]**

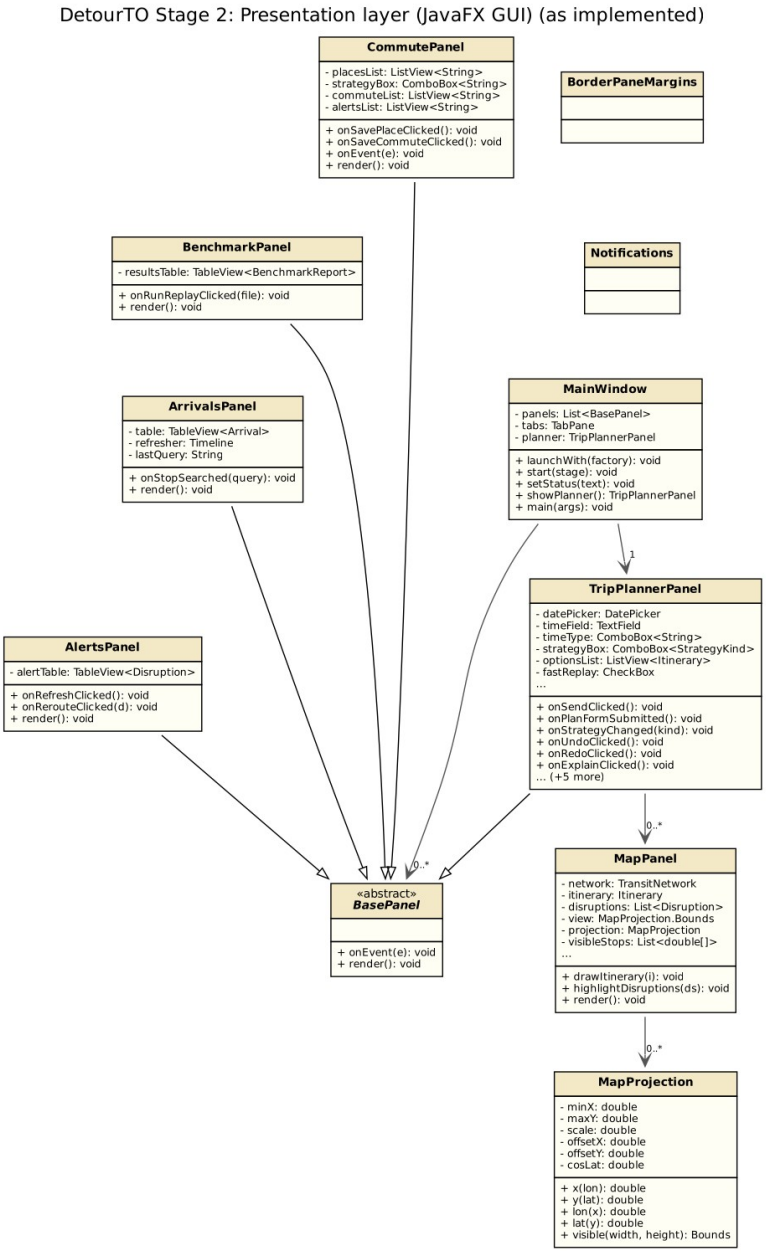
Appendix — class diagrams by layer (as implemented)

Full-resolution PNG and SVG files are in `docs/diagrams/stage2/`.

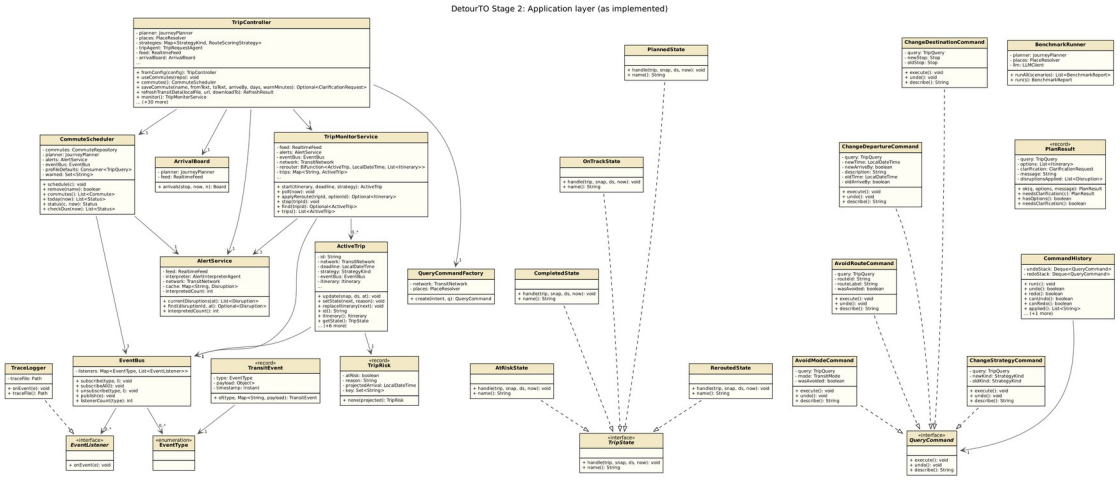
DetourTO Stage 2: Presentation layer (CLI) (as implemented)



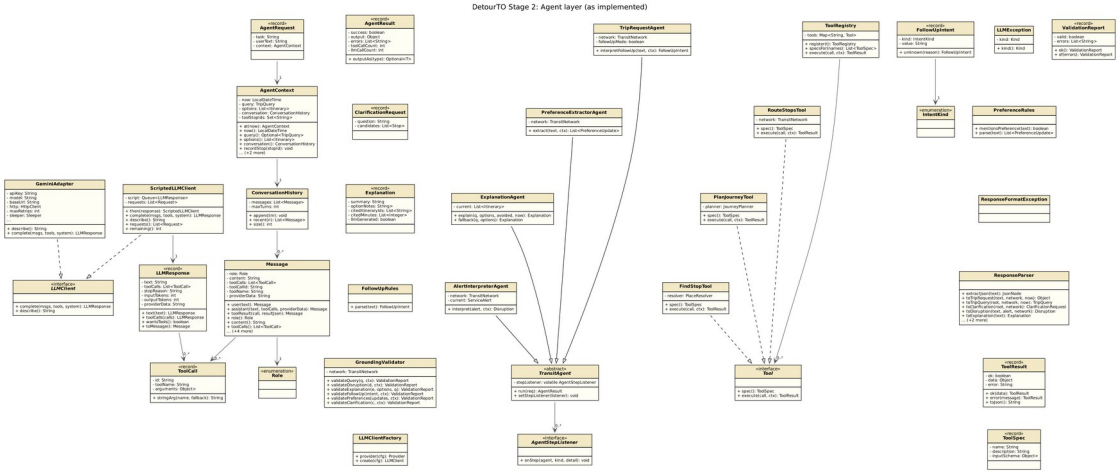
Presentation layer (CLI)



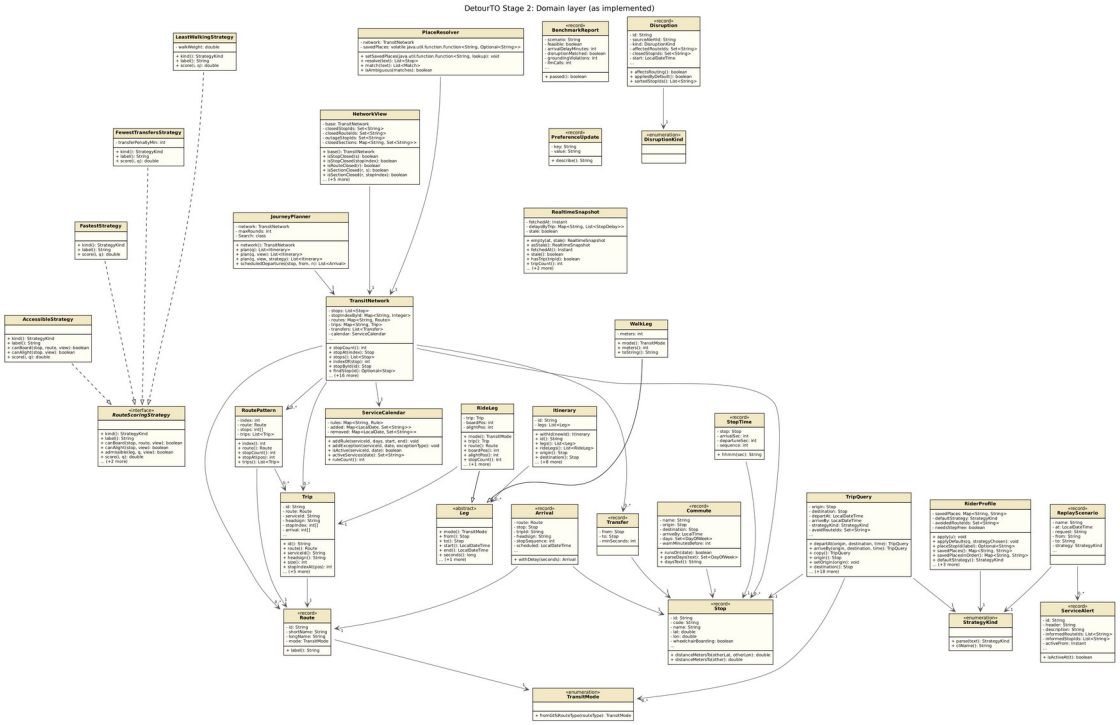
Presentation layer (JavaFX GUI)



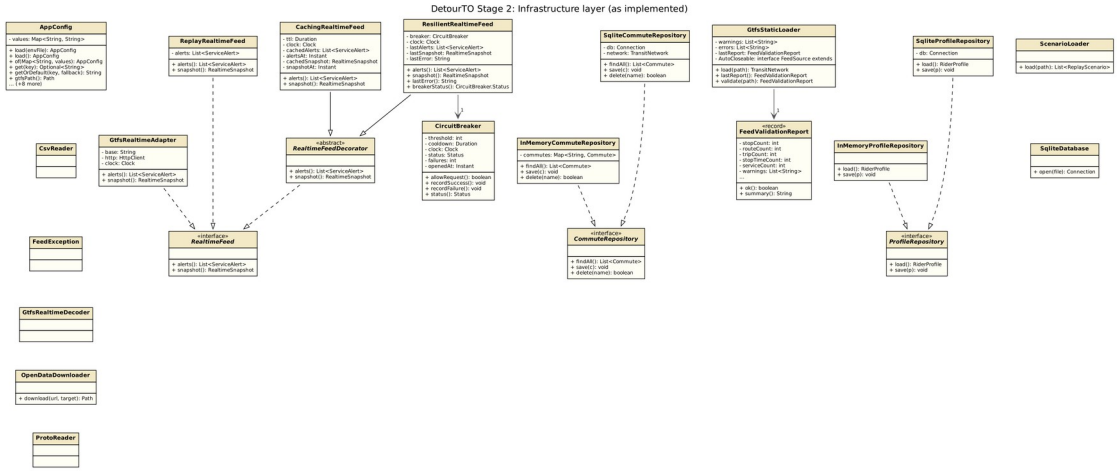
Application layer



Agent layer



Domain layer



Infrastructure layer